Apuração XML — código completo comentado

Aplicativo Streamlit que lê lotes de até 8.000 XMLs de NF-e/NFC-e (soltos ou em ZIP) e gera relatórios consolidados em Excel e PDF: soma por CFOP (com descrição), por mês e cliente, e notas detalhadas.

Executar: streamlit run app.py Dependências: streamlit, pandas, openpyxl, reportlab
Os comentários estão em verde; as definições de funções em azul. Linhas longas continuam na linha seguinte, marcadas com ↳. Emojis são mostrados como <U+XXXX> neste PDF (no código real são emojis).

Índice de funções e blocos principais

40	_CFOP_TABLE	descrições dos CFOPs mais comuns
217	cfop_description	Descrição oficial do CFOP ('' quando não catalogado).
264	local_name	Retorna o nome da tag XML sem o namespace.
269	first_text	Localiza a primeira ocorrência de uma tag pelo nome local.
281	to_float	
288	child_value	Valor numérico de um filho direto (pelo nome local).
296	group_sum	Soma <tag> dentro do grupo (ICMS, PIS, COFINS) do imposto do item.
308	parse_items	Extrai os itens (det) com CFOP, valor e tributos.
347	parse_xml	Extrai identificação, destinatário, totais e itens (CFOP) da NF-e/NFC-e.
452	collect_entries	Expande os envios (XML e ZIP) em uma lista de (nome, função_de_leitura).
488	process_uploads	Processa o lote (até MAX_FILES) com barra de progresso.
532	aggregate_reports	
552	aggregate_cfop	
590	build_excel	
637	fmt_doc	Formata CNPJ (00.000.000/0000-00) ou CPF (000.000.000-00).
647	emitentes_lista	Linhas 'Razão social — CNPJ: ...' dos emitentes com vendas a consumidor fin
665	_emitentes_pdf	
674	brl	
678	_table_style	
692	build_pdf	
778	Interface (Streamlit)	tela, upload, cache, tabelas e downloads

```
# ============================================================================
# Apuração XML - relatórios de NF-e / NFC-e em Excel e PDF (Streamlit)
#
# Como executar:   streamlit run app.py
# Dependências:    streamlit, pandas, openpyxl, reportlab
#
# Fluxo: o usuário envia XMLs (soltos ou em ZIP) -> o app lê cada nota e seus
# itens -> consolida por CFOP, mês e cliente -> mostra na tela e exporta
# os relatórios em Excel e PDF.
# ============================================================================

# --- Bibliotecas padrão: arquivos em memória, ZIP, leitura de XML, datas e decimais
import io
import zipfile
import xml.etree.ElementTree as ET
from datetime import datetime
from decimal import Decimal, InvalidOperation

# --- Bibliotecas de terceiros: tabelas (pandas), interface (streamlit) e PDF (reportlab)
import pandas as pd
import streamlit as st
from reportlab.lib import colors
from reportlab.lib.pagesizes import landscape, A4
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from xml.sax.saxutils import escape
from reportlab.lib.units import mm
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle

# Limite de arquivos XML processados por lote
MAX_FILES = 8000
# Colunas numéricas somadas nas consolidações por nota (vêm dos totais da nota)
METRICS = ["Faturamento (R$)", "ICMS (R$)", "PIS (R$)", "COFINS (R$)"]
# Colunas somadas por CFOP (calculadas item a item)
CFOP_METRICS = ["Valor Itens (R$)", "ICMS (R$)", "PIS (R$)", "COFINS (R$)"]

# Descrições dos CFOPs mais comuns (tabela oficial). As entradas 2xxx/3xxx usam o
# mesmo texto das 1xxx e as saídas 6xxx/7xxx o mesmo texto das 5xxx.
# CFOPs fora da tabela usam a "natureza da operação" (natOp) informada no XML.
# Para acrescentar um CFOP, basta incluir uma linha "codigo|descrição".
_CFOP_TABLE = """
1101|Compra para industrialização ou produção rural
1102|Compra para comercialização
1111|Compra para industrialização de mercadoria recebida anteriormente em consignação industrial
1113|Compra para comercialização, de mercadoria recebida anteriormente em consignação mercantil
1116|Compra para industrialização ou produção rural originada de encomenda para recebimento futuro
1117|Compra para comercialização originada de encomenda para recebimento futuro
1120|Compra para industrialização, em venda à ordem, já recebida do vendedor remetente
1121|Compra para comercialização, em venda à ordem, já recebida do vendedor remetente
1122|Compra para industrialização em que a mercadoria foi remetida pelo fornecedor ao industrializador sem transitar pelo est
↳abelecimento adquirente
1124|Industrialização efetuada por outra empresa
1125|Industrialização efetuada por outra empresa quando a mercadoria remetida para utilização no processo de industrialização
↳ não transitou pelo estabelecimento adquirente da mercadoria
1151|Transferência para industrialização ou produção rural
1152|Transferência para comercialização
1153|Transferência de energia elétrica para distribuição
1154|Transferência para utilização na prestação de serviço
1201|Devolução de venda de produção do estabelecimento
1202|Devolução de venda de mercadoria adquirida ou recebida de terceiros
1203|Devolução de venda de produção do estabelecimento, destinada à Zona Franca de Manaus ou Áreas de Livre Comércio
1204|Devolução de venda de mercadoria adquirida ou recebida de terceiros, destinada à Zona Franca de Manaus ou Áreas de Livre
↳ Comércio
1205|Anulação de valor relativo à prestação de serviço de comunicação
1206|Anulação de valor relativo à prestação de serviço de transporte
1207|Anulação de valor relativo à venda de energia elétrica
1208|Devolução de produção do estabelecimento, remetida em transferência
1209|Devolução de mercadoria adquirida ou recebida de terceiros, remetida em transferência
1251|Compra de energia elétrica para distribuição ou comercialização
1401|Compra para industrialização ou produção rural em operação com mercadoria sujeita ao regime de substituição tributária
1403|Compra para comercialização em operação com mercadoria sujeita ao regime de substituição tributária
1406|Compra de bem para o ativo imobilizado cuja mercadoria está sujeita ao regime de substituição tributária
1407|Compra de mercadoria para uso ou consumo cuja mercadoria está sujeita ao regime de substituição tributária
1408|Transferência para industrialização ou produção rural em operação com mercadoria sujeita ao regime de substituição tribu
↳tária
1409|Transferência para comercialização em operação com mercadoria sujeita ao regime de substituição tributária
1410|Devolução de venda de produção do estabelecimento em operação com produto sujeito ao regime de substituição tributária
1411|Devolução de venda de mercadoria adquirida ou recebida de terceiros em operação com mercadoria sujeita ao regime de subs
↳tituição tributária
1551|Compra de bem para o ativo imobilizado
1552|Transferência de bem do ativo imobilizado
1553|Devolução de venda de bem do ativo imobilizado
1554|Retorno de bem do ativo imobilizado remetido para uso fora do estabelecimento
1555|Entrada de bem do ativo imobilizado de terceiro, remetido para uso no estabelecimento
1556|Compra de material para uso ou consumo
1557|Transferência de material para uso ou consumo
1601|Recebimento, por transferência, de crédito de ICMS
1603|Ressarcimento de ICMS retido por substituição tributária
1901|Entrada para industrialização por encomenda
1902|Retorno de mercadoria remetida para industrialização por encomenda
1903|Entrada de mercadoria remetida para industrialização e não aplicada no referido processo
1904|Retorno de remessa para venda fora do estabelecimento
```

```
1905|Entrada de mercadoria recebida para depósito em depósito fechado ou armazém geral
1906|Retorno de mercadoria remetida para depósito fechado ou armazém geral
1907|Retorno simbólico de mercadoria remetida para depósito fechado ou armazém geral
1908|Entrada de bem por conta de contrato de comodato ou locação
1909|Retorno de bem remetido por conta de contrato de comodato ou locação
1910|Entrada de bonificação, doação ou brinde
1911|Entrada de amostra grátis
1912|Entrada de mercadoria ou bem recebido para demonstração
1913|Retorno de mercadoria ou bem remetido para demonstração
1914|Retorno de mercadoria ou bem remetido para exposição ou feira
1915|Entrada de mercadoria ou bem recebido para conserto ou reparo
1916|Retorno de mercadoria ou bem remetido para conserto ou reparo
1917|Entrada de mercadoria recebida em consignação mercantil ou industrial
1918|Devolução de mercadoria remetida em consignação mercantil ou industrial
1919|Devolução simbólica de mercadoria vendida ou utilizada em processo industrial, remetida anteriormente em consignação mercantil ou industrial
1920|Entrada de vasilhame ou sacaria
1921|Retorno de vasilhame ou sacaria
1922|Lançamento efetuado a título de simples faturamento decorrente de compra para recebimento futuro
1923|Entrada de mercadoria remetida por vendedor remetente, em venda à ordem
1926|Lançamento efetuado a título de reclassificação de mercadoria decorrente de formação de kit ou de sua desagregação
1932|Aquisição de serviço de transporte iniciado em unidade da Federação diversa daquela onde inscrito o prestador
1933|Aquisição de serviço tributado pelo ISSQN
1949|Outra entrada de mercadoria ou prestação de serviço não especificada
5101|Venda de produção do estabelecimento
5102|Venda de mercadoria adquirida ou recebida de terceiros
5103|Venda de produção do estabelecimento, efetuada fora do estabelecimento
5104|Venda de mercadoria adquirida ou recebida de terceiros, efetuada fora do estabelecimento
5105|Venda de produção do estabelecimento que não deva por ele transitar
5106|Venda de mercadoria adquirida ou recebida de terceiros, que não deva por ele transitar
5109|Venda de produção do estabelecimento destinada à Zona Franca de Manaus ou Áreas de Livre Comércio
5110|Venda de mercadoria adquirida ou recebida de terceiros, destinada à Zona Franca de Manaus ou Áreas de Livre Comércio
5111|Venda de produção do estabelecimento remetida anteriormente em consignação industrial
5112|Venda de mercadoria adquirida ou recebida de terceiros remetida anteriormente em consignação industrial
5113|Venda de produção do estabelecimento remetida anteriormente em consignação mercantil
5114|Venda de mercadoria adquirida ou recebida de terceiros remetida anteriormente em consignação mercantil
5115|Venda de mercadoria adquirida ou recebida de terceiros, recebida anteriormente em consignação industrial
5116|Venda de produção do estabelecimento originada de encomenda para entrega futura
5117|Venda de mercadoria adquirida ou recebida de terceiros, originada de encomenda para entrega futura
5118|Venda de produção do estabelecimento entregue ao destinatário por conta e ordem do adquirente originário, em venda à ordem
5119|Venda de mercadoria adquirida ou recebida de terceiros entregue ao destinatário por conta e ordem do adquirente originário, em venda à ordem
5120|Venda de mercadoria adquirida ou recebida de terceiros entregue ao destinatário pelo vendedor remetente, em venda à ordem
5124|Industrialização efetuada para outra empresa
5125|Industrialização efetuada para outra empresa quando a mercadoria remetida para utilização no processo de industrialização não transitou pelo estabelecimento adquirente da mercadoria
5151|Transferência de produção do estabelecimento
5152|Transferência de mercadoria adquirida ou recebida de terceiros
5153|Transferência de energia elétrica
5155|Transferência de produção do estabelecimento, que não deva por ele transitar
5156|Transferência de mercadoria adquirida ou recebida de terceiros, que não deva por ele transitar
5201|Devolução de compra para industrialização ou produção rural
5202|Devolução de compra para comercialização
5203|Devolução de mercadoria recebida em consignação mercantil ou industrial
5205|Anulação de valor relativo a aquisição de serviço de comunicação
5206|Anulação de valor relativo a aquisição de serviço de transporte
5207|Anulação de valor relativo à compra de energia elétrica
5208|Devolução de mercadoria recebida em transferência para industrialização ou produção rural
5209|Devolução de mercadoria recebida em transferência para comercialização
5210|Devolução de compra para utilização na prestação de serviço
5251|Venda de energia elétrica para distribuição ou comercialização
5301|Prestação de serviço de comunicação para execução de serviço da mesma natureza
5307|Prestação de serviço de comunicação a não contribuinte
5351|Prestação de serviço de transporte para execução de serviço da mesma natureza
5352|Prestação de serviço de transporte a estabelecimento industrial
5353|Prestação de serviço de transporte a estabelecimento comercial
5357|Prestação de serviço de transporte a não contribuinte
5359|Prestação de serviço de transporte a contribuinte ou a não contribuinte quando a mercadoria transportada está dispensada de emissão de nota fiscal
5401|Venda de produção do estabelecimento em operação com produto sujeito ao regime de substituição tributária, na condição de contribuinte substituto
5402|Venda de produção do estabelecimento de produto sujeito ao regime de substituição tributária, em operação entre contribuintes substitutos do mesmo produto
5403|Venda de mercadoria adquirida ou recebida de terceiros em operação com mercadoria sujeita ao regime de substituição tributária, na condição de contribuinte substituto
5404|Venda de mercadoria sujeita ao regime de substituição tributária, cujo imposto já tenha sido retido anteriormente
5405|Venda de mercadoria adquirida ou recebida de terceiros em operação com mercadoria sujeita ao regime de substituição tributária, na condição de contribuinte substituído
5408|Transferência de produção do estabelecimento em operação com produto sujeito ao regime de substituição tributária
5409|Transferência de mercadoria adquirida ou recebida de terceiros em operação com mercadoria sujeita ao regime de substituição tributária
5410|Devolução de compra para industrialização ou produção rural em operação com mercadoria sujeita ao regime de substituição tributária
5411|Devolução de compra para comercialização em operação com mercadoria sujeita ao regime de substituição tributária
5412|Devolução de bem do ativo imobilizado, em operação com mercadoria sujeita ao regime de substituição tributária
5413|Devolução de mercadoria destinada ao uso ou consumo, em operação com mercadoria sujeita ao regime de substituição tributária
5414|Remessa de produção do estabelecimento para venda fora do estabelecimento em operação com produto sujeito ao regime de substituição tributária
```

```
5415|Remessa de mercadoria adquirida ou recebida de terceiros para venda fora do estabelecimento, em operação com mercadoria sujeita ao regime de substituição tributária
5451|Remessa de animal e de insumo para estabelecimento produtor
5501|Remessa de produção do estabelecimento, com fim específico de exportação
5502|Remessa de mercadoria adquirida ou recebida de terceiros, com fim específico de exportação
5503|Devolução de mercadoria recebida com fim específico de exportação
5551|Venda de bem do ativo imobilizado
5552|Transferência de bem do ativo imobilizado
5553|Devolução de compra de bem para o ativo imobilizado
5554|Remessa de bem do ativo imobilizado para uso fora do estabelecimento
5555|Devolução de bem do ativo imobilizado de terceiro, recebido para uso no estabelecimento
5556|Devolução de compra de material de uso ou consumo
5557|Transferência de material de uso ou consumo
5601|Transferência de crédito de ICMS acumulado
5603|Ressarcimento de ICMS retido por substituição tributária
5901|Remessa para industrialização por encomenda
5902|Retorno de mercadoria utilizada na industrialização por encomenda
5903|Retorno de mercadoria recebida para industrialização e não aplicada no referido processo
5904|Remessa para venda fora do estabelecimento
5905|Remessa para depósito fechado ou armazém geral
5906|Retorno de mercadoria depositada em depósito fechado ou armazém geral
5907|Retorno simbólico de mercadoria depositada em depósito fechado ou armazém geral
5908|Remessa de bem por conta de contrato de comodato ou locação
5909|Retorno de bem recebido por conta de contrato de comodato ou locação
5910|Remessa em bonificação, doação ou brinde
5911|Remessa de amostra grátis
5912|Remessa de mercadoria ou bem para demonstração
5913|Retorno de mercadoria ou bem recebido para demonstração
5914|Remessa de mercadoria ou bem para exposição ou feira
5915|Remessa de mercadoria ou bem para conserto ou reparo
5916|Retorno de mercadoria ou bem recebido para conserto ou reparo
5917|Remessa de mercadoria em consignação mercantil ou industrial
5918|Devolução de mercadoria recebida em consignação mercantil ou industrial
5919|Devolução simbólica de mercadoria vendida ou utilizada em processo industrial, recebida anteriormente em consignação mercantil ou industrial
5920|Remessa de vasilhame ou sacaria
5921|Devolução de vasilhame ou sacaria
5922|Lançamento efetuado a título de simples faturamento decorrente de venda para entrega futura
5923|Remessa de mercadoria por conta e ordem de terceiros, em venda à ordem ou em operações com armazém geral ou depósito fechado
5926|Lançamento efetuado a título de reclassificação de mercadoria decorrente de formação de kit ou de sua desagregação
5927|Lançamento efetuado a título de baixa de estoque decorrente de perda, roubo ou deterioração
5928|Lançamento efetuado a título de baixa de estoque decorrente do encerramento da atividade da empresa
5929|Lançamento efetuado em decorrência de emissão de documento fiscal relativo a operação ou prestação também registrada em equipamento Emissor de Cupom Fiscal (ECF)
5932|Prestação de serviço de transporte iniciada em unidade da Federação diversa daquela onde inscrito o prestador
5933|Prestação de serviço tributado pelo ISSQN (Nota Fiscal conjugada)
5949|Outra saída de mercadoria ou prestação de serviço não especificado
"""
CFOP_DESC = dict(line.split("|", 1) for line in _CFOP_TABLE.strip().splitlines())
_CFOP_OVERRIDES = {
    "6107": "Venda de produção do estabelecimento, destinada a não contribuinte",
    "6108": "Venda de mercadoria adquirida ou recebida de terceiros, destinada a não contribuinte",
}
_CFOP_BASE = {"1": "1", "2": "1", "3": "1", "5": "5", "6": "5", "7": "5"}


def cfop_description(cfop):
    """Descrição oficial do CFOP ('' quando não catalogado)."""
    c = str(cfop).strip()
    if c in _CFOP_OVERRIDES:
        return _CFOP_OVERRIDES[c]
    if len(c) == 4 and c.isdigit() and c[0] in _CFOP_BASE:
        return CFOP_DESC.get(_CFOP_BASE[c[0]] + c[1:], "")
    return ""

# Configuração da página: título da aba, ícone e layout largo
st.set_page_config(
    page_title="Apuração XML | Relatórios",
    page_icon="<U+1F4C4>",
    layout="wide",
)

# CSS personalizado: fundo, cartões de métricas e faixa de cabeçalho (hero)
st.markdown("""
<style>
    .main {background: #f6f8fb;}
    .block-container {padding-top: 2rem; padding-bottom: 2rem;}
    div[data-testid="stMetric"] {
        background: white; border: 1px solid #e7ebf1;
        padding: 16px; border-radius: 12px;
    }
    .hero {
        padding: 22px 26px; border-radius: 16px;
        background: linear-gradient(110deg,#12345a,#176b87);
        color: white; margin-bottom: 18px;
    }
    .hero h1 {color:white; margin:0;}
    .hero p {color:#e7f4fa; margin:6px 0 0 0;}
</style>
""", unsafe_allow_html=True)
```

```
251
252 # Faixa de cabeçalho exibida no topo da página
253 st.markdown("""
254 <div class="hero">
255   <h1><U+1F4C4> Apuração XML</h1>
256   <p>Importe lotes de até 8.000 arquivos NF-e/NFC-e (XML ou ZIP) e gere relatórios consolidados em Excel e PDF, inclusive por
    ↳ CFOP.</p>
257 </div>
258 """, unsafe_allow_html=True)
259
260
261 # ---------------------------------------------------------------------------
262 # Leitura dos XMLs
263 # ---------------------------------------------------------------------------
264 def local_name(tag):
265     """Retorna o nome da tag XML sem o namespace."""
266     return tag.rsplit("}", 1)[-1].split(":")[-1]
267
268
269 def first_text(root, tag_names, default=""):
270     """Localiza a primeira ocorrência de uma tag pelo nome local."""
271     wanted = set(tag_names if isinstance(tag_names, (list, tuple, set)) else [tag_names])
272     for node in root.iter():
273         if local_name(node.tag) in wanted and node.text:
274             value = node.text.strip()
275             if value:
276                 return value
277     return default
278
279
280 # Converte texto em número (aceita vírgula decimal); devolve 0.0 se não der para converter
281 def to_float(value):
282     try:
283         return float(Decimal(str(value or "0").replace(",", ".")))
284     except (InvalidOperation, ValueError, TypeError):
285         return 0.0
286
287
288 def child_value(node, tag):
289     """Valor numérico de um filho direto (pelo nome local)."""
290     for child in node:
291         if local_name(child.tag) == tag:
292             return to_float(child.text)
293     return 0.0
294
295
296 def group_sum(imposto, group, tag):
297     """Soma <tag> dentro do grupo (ICMS, PIS, COFINS) do imposto do item.
298     Grupos de substituição (PISST/COFINSST) não entram, pois têm outro nome."""
299     total = 0.0
300     for grp in imposto:
301         if local_name(grp.tag) == group:
302             for node in grp.iter():
303                 if local_name(node.tag) == tag:
304                     total += to_float(node.text)
305     return total
306
307
308 def parse_items(root, arquivo, chave, numero, mes_ano, natop=""):
309     """Extrai os itens (det) com CFOP, valor e tributos."""
310     items = []
311     # Cada <det> é um item da nota; o CFOP fica em det/prod/CFOP
312     for det in root.iter():
313         if local_name(det.tag) != "det":
314             continue
315         prod = next((n for n in det if local_name(n.tag) == "prod"), None)
316         if prod is None:
317             continue
318         imposto = next((n for n in det if local_name(n.tag) == "imposto"), None)
319         cfop = "N/A"
320         for n in prod:
321             if local_name(n.tag) == "CFOP" and n.text and n.text.strip():
322                 cfop = n.text.strip()
323                 break
324         # Valor do item = produto + frete + seguro + outras despesas - desconto
325         valor = (
326             child_value(prod, "vProd") + child_value(prod, "vFrete")
327             + child_value(prod, "vSeg") + child_value(prod, "vOutro")
328             - child_value(prod, "vDesc")
329         )
330         # Linha do item; impostos lidos só dentro dos grupos ICMS, PIS e COFINS (PISST/COFINSST ficam de fora)
331         items.append({
332             "Arquivo": arquivo,
333             "Chave": chave,
334             "Número": numero,
335             "Mês/Ano": mes_ano,
336             "CFOP": cfop,
337             "Descrição CFOP": cfop_description(cfop),
338             "Natureza Operação": natop,
339             "Valor Itens (R$)": valor,
340             "ICMS (R$)": group_sum(imposto, "ICMS", "vICMS") if imposto is not None else 0.0,
```

```
            "PIS (R$)": group_sum(imposto, "PIS", "vPIS") if imposto is not None else 0.0,
            "COFINS (R$)": group_sum(imposto, "COFINS", "vCOFINS") if imposto is not None else 0.0,
        })
    return items


def parse_xml(name, content):
    """Extrai identificação, destinatário, totais e itens (CFOP) da NF-e/NFC-e.
    Retorna (nota, itens, erro)."""
    try:
        # Lê o XML (NF-e ou NFC-e, com ou sem o envelope nfeProc)
        root = ET.fromstring(content)

        # Grupo <ide>: identificação da nota (modelo, número, data, natureza da operação)
        ide = next((n for n in root.iter() if local_name(n.tag) == "ide"), None)
        if ide is None:
            raise ValueError("Não foi encontrado o grupo de identificação <ide>.")

        def ide_value(tag, default=""):
            for node in ide:
                if local_name(node.tag) == tag and node.text:
                    return node.text.strip()
            return default

        # Modelo 55 = NF-e; modelo 65 = NFC-e
        modelo_cod = ide_value("mod", "N/A")
        modelo = {"55": "NF-e", "65": "NFC-e"}.get(modelo_cod, f"Modelo {modelo_cod}")
        numero = ide_value("nNF", "N/A")
        # Data de emissão (dhEmi na versão 4.0; dEmi em versões antigas); Mês/Ano = AAAA-MM
        dh_emi = ide_value("dhEmi") or ide_value("dEmi") or "N/A"
        data_emissao = dh_emi[:10] if dh_emi != "N/A" else "N/A"
        mes_ano = data_emissao[:7] if data_emissao != "N/A" else "N/A"

        # Chave de acesso (usada para evitar contagem em duplicidade)
        chave = ""
        inf = next((n for n in root.iter() if local_name(n.tag) == "infNFe"), None)
        if inf is not None:
            chave = (inf.get("Id") or "").replace("NFe", "").strip()
        if not chave:
            chave = first_text(root, "chNFe", "")

        # Destinatário (cliente): CNPJ/CPF e nome; se não houver, fica 'Consumidor Final'
        dest = next((n for n in root.iter() if local_name(n.tag) == "dest"), None)
        doc_cliente = "Não identificado"
        nome_cliente = "Consumidor Final"
        if dest is not None:
            for node in dest.iter():
                tag = local_name(node.tag)
                if tag in ("CNPJ", "CPF") and node.text:
                    doc_cliente = node.text.strip()
                elif tag == "xNome" and node.text:
                    nome_cliente = node.text.strip()

        # Emitente: CNPJ/CPF e razão social de quem emitiu a nota
        emit = next((n for n in root.iter() if local_name(n.tag) == "emit"), None)
        cnpj_emit = ""
        nome_emit = ""
        if emit is not None:
            for node in emit:
                tag = local_name(node.tag)
                if tag in ("CNPJ", "CPF") and node.text and not cnpj_emit:
                    cnpj_emit = node.text.strip()
                elif tag == "xNome" and node.text:
                    nome_emit = node.text.strip()

        # Venda a consumidor final: indFinal = 1 (ou NFC-e, que é sempre consumidor final)
        consumidor_final = ide_value("indFinal") == "1" or modelo_cod == "65"
        if not consumidor_final:
            cnpj_emit = ""
            nome_emit = ""

        # Totais da nota (<ICMSTot>); se o grupo faltar, procura as tags em qualquer ponto do XML
        total = next((n for n in root.iter() if local_name(n.tag) == "ICMSTot"), None)
        values = {"vNF": 0, "vICMS": 0, "vPIS": 0, "vCOFINS": 0}
        if total is not None:
            for node in total.iter():
                tag = local_name(node.tag)
                if tag in values:
                    values[tag] = to_float(node.text)
        else:
            for tag in values:
                values[tag] = to_float(first_text(root, tag, "0"))

        # Linha da nota para o relatório detalhado
        nota = {
            "Arquivo": name,
            "Chave": chave,
            "Consumidor Final": "Sim" if consumidor_final else "Não",
            "CNPJ Emitente": cnpj_emit,
            "Emitente": nome_emit,
            "Modelo": modelo,
```

```
            "Número": numero,
            "Data Emissão": data_emissao,
            "Mês/Ano": mes_ano,
            "Documento Cliente": doc_cliente,
            "Nome Cliente": nome_cliente,
            "Faturamento (R$)": values["vNF"],
            "ICMS (R$)": values["vICMS"],
            "PIS (R$)": values["vPIS"],
            "COFINS (R$)": values["vCOFINS"],
        }
        # Itens da nota (base da soma por CFOP)
        itens = parse_items(root, name, chave, numero, mes_ano, ide_value("natOp"))
        return nota, itens, None
    # Erros viram mensagens para o usuário, sem derrubar o aplicativo
    except ET.ParseError as exc:
        return None, [], f"XML inválido ou malformado: {exc}"
    except Exception as exc:
        return None, [], str(exc)


def collect_entries(uploads, ignorar_recebidos=False):
    """Expande os envios (XML e ZIP) em uma lista de (nome, função_de_leitura).
    Com ignorar_recebidos, descarta XMLs cujo nome (ou pasta dentro do ZIP) contém 'recebidos'.
    Retorna (entradas, quantidade_ignorada)."""
    entries = []
    ignorados = 0

    def ignorar(nome):
        return ignorar_recebidos and "recebidos" in nome.lower()

    # Percorre cada envio: XML solto ou ZIP com vários XMLs
    for up in uploads:
        if up.name.lower().endswith(".zip"):
            try:
                zf = zipfile.ZipFile(io.BytesIO(up.getvalue()))
            except zipfile.BadZipFile:
                entries.append((up.name, None))
                continue
            for info in zf.infolist():
                low = info.filename.lower()
                # Descarta pastas, lixo do macOS (__MACOSX) e arquivos que não são XML
                if info.is_dir() or "__macosx" in low or not low.endswith(".xml"):
                    continue
                if ignorar(info.filename):
                    ignorados += 1
                    continue
                # A leitura fica adiada (lambda) para não carregar todos os XMLs na memória de uma vez
                entries.append((info.filename, lambda zf=zf, info=info: zf.read(info)))
        else:
            if ignorar(up.name):
                ignorados += 1
                continue
            entries.append((up.name, lambda up=up: up.getvalue()))
    return entries, ignorados


def process_uploads(uploads, ignorar_recebidos=False):
    """Processa o lote (até MAX_FILES) com barra de progresso.
    Retorna (df, df_itens, errors, duplicates, total_found, ignorados)."""
    entries, ignorados = collect_entries(uploads, ignorar_recebidos)
    total_found = len(entries)
    entries = entries[:MAX_FILES]

    # notas: uma linha por nota | itens: uma linha por item | seen: chaves de acesso já vistas
    notas, itens, errors = [], [], []
    seen = set()
    duplicates = 0
    progress = st.progress(0.0, text="Lendo arquivos...")
    n = len(entries)
    # Lê um arquivo por vez e atualiza a barra de progresso a cada 50 arquivos
    for i, (name, loader) in enumerate(entries, start=1):
        if loader is None:
            errors.append((name, "ZIP inválido ou corrompido."))
        else:
            try:
                nota, its, error = parse_xml(name, loader())
            except Exception as exc:
                nota, its, error = None, [], str(exc)
            if nota is None:
                errors.append((name, error))
            # Mesma chave de acesso = nota duplicada; só a primeira é contada
            elif nota["Chave"] and nota["Chave"] in seen:
                duplicates += 1
            else:
                if nota["Chave"]:
                    seen.add(nota["Chave"])
                notas.append(nota)
                itens.extend(its)
        if i % 50 == 0 or i == n:
            progress.progress(i / max(n, 1), text=f"Lendo arquivos... {i:,}/{n:,}".replace(",", "."))
    progress.empty()
```

```
    df = pd.DataFrame(notas)
    df_itens = pd.DataFrame(itens)
    return df, df_itens, errors, duplicates, total_found, ignorados


# ---------------------------------------------------------------------------
# Consolidações
# ---------------------------------------------------------------------------
def aggregate_reports(df):
    # Três consolidações: por mês e cliente, por mês e por cliente
    by_month_client = (
        df.groupby(["Mês/Ano", "Documento Cliente", "Nome Cliente"], dropna=False)
        .agg(**{m: (m, "sum") for m in METRICS}, **{"Qtd Notas": ("Número", "count")})
        .reset_index()
    )
    by_month = (
        df.groupby(["Mês/Ano"], dropna=False)
        .agg(**{m: (m, "sum") for m in METRICS}, **{"Qtd Notas": ("Número", "count")})
        .reset_index()
    )
    by_client = (
        df.groupby(["Documento Cliente", "Nome Cliente"], dropna=False)
        .agg(**{m: (m, "sum") for m in METRICS}, **{"Qtd Notas": ("Número", "count")})
        .reset_index()
    )
    return by_month_client, by_month, by_client


def aggregate_cfop(df_itens):
    cols = ["CFOP", "Descrição CFOP", *CFOP_METRICS, "Qtd Itens", "Qtd Notas"]
    cols_m = ["Mês/Ano", *cols]
    if df_itens.empty:
        return pd.DataFrame(columns=cols), pd.DataFrame(columns=cols_m)
    spec = {
        **{m: (m, "sum") for m in CFOP_METRICS},
        "Qtd Itens": ("CFOP", "count"),
        "Qtd Notas": ("Arquivo", "nunique"),
    }

    # Descrição: tabela oficial; se o CFOP não estiver catalogado, usa a natureza
    # da operação mais frequente informada nos XMLs.
    def natop_mais_frequente(s):
        s = s[s.astype(str).str.strip() != ""]
        return s.mode().iat[0] if not s.empty else ""

    natop = df_itens.groupby("CFOP")["Natureza Operação"].agg(natop_mais_frequente)
    desc = {}
    for cfop in df_itens["CFOP"].unique():
        d = cfop_description(cfop)
        if not d:
            n = natop.get(cfop, "")
            d = f"{n} (natureza da operação no XML)" if n else "CFOP não catalogado"
        desc[cfop] = d

    by_cfop = df_itens.groupby("CFOP", dropna=False).agg(**spec).reset_index()
    by_cfop.insert(1, "Descrição CFOP", by_cfop["CFOP"].map(desc))
    by_cfop = by_cfop.sort_values("Valor Itens (R$)", ascending=False).reset_index(drop=True)
    by_month_cfop = df_itens.groupby(["Mês/Ano", "CFOP"], dropna=False).agg(**spec).reset_index()
    by_month_cfop.insert(2, "Descrição CFOP", by_month_cfop["CFOP"].map(desc))
    df_itens["Descrição CFOP"] = df_itens["CFOP"].map(desc)
    return by_cfop, by_month_cfop


# ---------------------------------------------------------------------------
# Exportação
# ---------------------------------------------------------------------------
def build_excel(df, by_month_client, by_month, by_client, by_cfop, by_month_cfop, df_itens):
    output = io.BytesIO()
    # Cada consolidação vira uma aba do Excel
    with pd.ExcelWriter(output, engine="openpyxl") as writer:
        by_cfop.to_excel(writer, sheet_name="Soma por CFOP", index=False)
        by_month_cfop.to_excel(writer, sheet_name="CFOP por Mês", index=False)
        by_month_client.to_excel(writer, sheet_name="Mês e Cliente", index=False)
        by_month.to_excel(writer, sheet_name="Consolidado Mensal", index=False)
        by_client.to_excel(writer, sheet_name="Consolidado Cliente", index=False)
        df.to_excel(writer, sheet_name="Notas Detalhadas", index=False)
        df_itens.to_excel(writer, sheet_name="Itens por CFOP", index=False)

        # Linha de total na aba de CFOP
        ws_cfop = writer.book["Soma por CFOP"]
        if len(by_cfop):
            last = len(by_cfop) + 1
            total_row = last + 1
            ws_cfop.cell(row=total_row, column=1, value="TOTAL")
            for idx, col in enumerate(by_cfop.columns, start=1):
                if col not in ("CFOP", "Descrição CFOP"):
                    letter = ws_cfop.cell(row=1, column=idx).column_letter
                    ws_cfop.cell(row=total_row, column=idx, value=f"=SUM({letter}2:{letter}{last})")
                    ws_cfop.cell(row=total_row, column=idx).font = ws_cfop.cell(row=1, column=1).font.copy(bold=True)
            ws_cfop.cell(row=total_row, column=1).font = ws_cfop.cell(row=1, column=1).font.copy(bold=True)
```

```
614
615         # Formatação de cada aba: cabeçalho congelado, filtro, largura das colunas e formato R$
616         for ws in writer.book.worksheets:
617             ws.freeze_panes = "A2"
618             ws.auto_filter.ref = ws.dimensions
619             money_cols = {c[0].column for c in ws.iter_cols(min_row=1, max_row=1)
620                           if "(R$)" in str(c[0].value or "")}
621             for cells in ws.columns:
622                 sample = cells[:2000]
623                 max_len = max((len(str(c.value or "")) for c in sample), default=10)
624                 cap = 80 if str(cells[0].value) == "Descrição CFOP" else 38
625                 ws.column_dimensions[cells[0].column_letter].width = min(max(max_len + 2, 12), cap)
626             for row in ws.iter_rows(min_row=2):
627                 for cell in row:
628                     if cell.column in money_cols and (
629                         isinstance(cell.value, (int, float))
630                         or (isinstance(cell.value, str) and cell.value.startswith("="))
631                     ):
632                         cell.number_format = '"R$" #,##0.00'
633     output.seek(0)
634     return output.getvalue()
635
636
637 def fmt_doc(doc):
638     """Formata CNPJ (00.000.000/0000-00) ou CPF (000.000.000-00)."""
639     d = str(doc or "").strip()
640     if len(d) == 14 and d.isdigit():
641         return f"{d[:2]}.{d[2:5]}.{d[5:8]}/{d[8:12]}-{d[12:]}"
642     if len(d) == 11 and d.isdigit():
643         return f"{d[:3]}.{d[3:6]}.{d[6:9]}-{d[9:]}"
644     return d
645
646
647 def emitentes_lista(df, limite=5):
648     """Linhas 'Razão social — CNPJ: ...' dos emitentes com vendas a consumidor final."""
649     if df.empty or "CNPJ Emitente" not in df:
650         return []
651     df = df[df["CNPJ Emitente"].astype(str).str.strip() != ""]
652     if df.empty:
653         return []
654     pares = (df[["CNPJ Emitente", "Emitente"]].drop_duplicates()
655              .sort_values("CNPJ Emitente").values.tolist())
656     linhas = []
657     for cnpj, nome in pares[:limite]:
658         doc = fmt_doc(cnpj) or "não informado"
659         linhas.append(f"{nome or 'Emitente'} — CNPJ: {doc}")
660     if len(pares) > limite:
661         linhas.append(f"... e mais {len(pares) - limite} emitente(s)")
662     return linhas
663
664
665 def _emitentes_pdf(df):
666     linhas = emitentes_lista(df)
667     if not linhas:
668         return "Emitente (vendas a consumidor final): nenhuma venda a consumidor final no lote"
669     rotulo = "Emitente (vendas a consumidor final): " if len(linhas) == 1 else \
670         "Emitentes (vendas a consumidor final):<br/>"
671     return rotulo + "<br/>".join(escape(l) for l in linhas)
672
673
674 def brl(value):
675     return f"R$ {value:,.2f}".replace(",", "X").replace(".", ",").replace("X", ".")
676
677
678 def _table_style(font_size=None):
679     style = [
680         ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#164e70")),
681         ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
682         ("GRID", (0, 0), (-1, -1), .35, colors.HexColor("#cbd5e1")),
683         ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
684         ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, colors.HexColor("#f1f5f9")]),
685         ("PADDING", (0, 0), (-1, -1), 4),
686     ]
687     if font_size:
688         style.append(("FONTSIZE", (0, 0), (-1, -1), font_size))
689     return TableStyle(style)
690
691
692 def build_pdf(df, by_month_client, by_cfop):
693     output = io.BytesIO()
694     # Documento A4 em paisagem com margens de 12 mm
695     doc = SimpleDocTemplate(
696         output, pagesize=landscape(A4),
697         rightMargin=12*mm, leftMargin=12*mm, topMargin=12*mm, bottomMargin=12*mm
698     )
699     styles = getSampleStyleSheet()
700     story = [
701         Paragraph("Relatório de Apuração Fiscal — NF-e / NFC-e", styles["Title"]),
702         Paragraph(f"Relatório gerado em {datetime.now().strftime('%d/%m/%Y %H:%M')}", styles["Normal"]),
703         Paragraph(_emitentes_pdf(df), styles["Normal"]),
704         Spacer(1, 5*mm),
```

```
    ]
    # Quadro de indicadores gerais do topo do PDF
    totals = [
        ["Indicador", "Total"],
        ["Quantidade de notas", str(len(df))],
        ["Faturamento", brl(df["Faturamento (R$)"].sum())],
        ["ICMS", brl(df["ICMS (R$)"].sum())],
        ["PIS", brl(df["PIS (R$)"].sum())],
        ["COFINS", brl(df["COFINS (R$)"].sum())],
    ]
    t = Table(totals, colWidths=[55*mm, 45*mm])
    t.setStyle(TableStyle([
        ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#164e70")),
        ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
        ("GRID", (0, 0), (-1, -1), .4, colors.HexColor("#cbd5e1")),
        ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, colors.HexColor("#f1f5f9")]),
        ("PADDING", (0, 0), (-1, -1), 6),
    ]))
    story += [t, Spacer(1, 7*mm), Paragraph("Soma dos valores por CFOP", styles["Heading2"])]

    # --- Soma por CFOP ---
    desc_style = ParagraphStyle("cfop_desc", parent=styles["Normal"], fontSize=7, leading=8.5)
    cfop_data = [["CFOP", "Descrição", "Notas", "Itens", "Valor dos itens", "ICMS", "PIS", "COFINS"]]
    for _, r in by_cfop.iterrows():
        cfop_data.append([
            str(r["CFOP"]), Paragraph(escape(str(r["Descrição CFOP"])), desc_style),
            str(int(r["Qtd Notas"])), str(int(r["Qtd Itens"])),
            brl(r["Valor Itens (R$)"]), brl(r["ICMS (R$)"]),
            brl(r["PIS (R$)"]), brl(r["COFINS (R$)"]),
        ])
    if len(by_cfop):
        cfop_data.append([
            "TOTAL", "", "", str(int(by_cfop["Qtd Itens"].sum())),
            brl(by_cfop["Valor Itens (R$)"].sum()), brl(by_cfop["ICMS (R$)"].sum()),
            brl(by_cfop["PIS (R$)"].sum()), brl(by_cfop["COFINS (R$)"].sum()),
        ])
    cfop_table = Table(cfop_data, repeatRows=1,
                       colWidths=[16*mm, 92*mm, 14*mm, 14*mm, 34*mm, 30*mm, 30*mm, 30*mm])
    cfop_style = _table_style(8)
    if len(by_cfop):
        cfop_style.add("FONTNAME", (0, -1), (-1, -1), "Helvetica-Bold")
        cfop_style.add("BACKGROUND", (0, -1), (-1, -1), colors.HexColor("#e2e8f0"))
    cfop_table.setStyle(cfop_style)
    story += [
        cfop_table,
        Paragraph(
            "Valor dos itens = vProd + frete + seguro + outras despesas − desconto, somado por CFOP "
            "do item. Pode diferir do faturamento (vNF) quando há IPI, ICMS-ST ou outros acréscimos.",
            styles["Italic"],
        ),
        Spacer(1, 7*mm),
        Paragraph("Consolidado por mês e cliente", styles["Heading2"]),
    ]

    # --- Mês e cliente ---
    data = [["Mês/Ano", "Documento", "Cliente", "Notas",
             "Faturamento", "ICMS", "PIS", "COFINS"]]
    for _, row in by_month_client.iterrows():
        data.append([
            str(row["Mês/Ano"]), str(row["Documento Cliente"]),
            str(row["Nome Cliente"])[:32], str(int(row["Qtd Notas"])),
            brl(row["Faturamento (R$)"]), brl(row["ICMS (R$)"]),
            brl(row["PIS (R$)"]), brl(row["COFINS (R$)"]),
        ])
    table = Table(data, repeatRows=1, colWidths=[22*mm, 32*mm, 54*mm, 15*mm, 32*mm, 25*mm, 25*mm, 25*mm])
    table.setStyle(_table_style(7))
    story.append(table)
    doc.build(story)
    output.seek(0)
    return output.getvalue()


# ---------------------------------------------------------------------------
# Interface
# ---------------------------------------------------------------------------
# Opção de tela: pular XMLs que tenham "recebidos" no nome
ignorar_recebidos = st.checkbox(
    "Não ler arquivos XML com \"recebidos\" no nome",
    value=False,
    help="Ignora XMLs cujo nome contenha \"recebidos\" (em qualquer combinação de maiúsculas/minúsculas). "
         "Em arquivos ZIP, vale também para o nome da pasta dentro do ZIP.",
)

uploads = st.file_uploader(
    f"Selecione arquivos XML de NF-e/NFC-e ou um/mais ZIPs (até {MAX_FILES:,} XMLs por lote)".replace(",", "."),
    type=["xml", "zip"], accept_multiple_files=True,
    help="Para lotes grandes, compacte os XMLs em um arquivo .zip e envie-o: é bem mais rápido."
)

# Fluxo principal: só processa depois que houver arquivos enviados
if uploads:
```

```
    # Assinatura do envio + opção marcada: evita reprocessar os XMLs a cada clique na tela
    signature = (tuple((u.name, u.size) for u in uploads), ignorar_recebidos)
    cached = st.session_state.get("result")
    if cached and cached["signature"] == signature:
        df, df_itens, errors, duplicates, total_found, ignorados = (
            cached["df"], cached["df_itens"], cached["errors"],
            cached["duplicates"], cached["total_found"], cached["ignorados"],
        )
        by_month_client, by_month, by_client = cached["agg"]
        by_cfop, by_month_cfop = cached["cfop"]
        xlsx, pdf = cached["xlsx"], cached["pdf"]
    else:
        df, df_itens, errors, duplicates, total_found, ignorados = process_uploads(
            uploads, ignorar_recebidos)
        if df.empty:
            by_month_client = by_month = by_client = None
            by_cfop = by_month_cfop = None
            xlsx = pdf = None
        else:
            by_month_client, by_month, by_client = aggregate_reports(df)
            by_cfop, by_month_cfop = aggregate_cfop(df_itens)
            with st.spinner("Gerando Excel e PDF..."):
                xlsx = build_excel(df, by_month_client, by_month, by_client,
                                   by_cfop, by_month_cfop, df_itens)
                pdf = build_pdf(df, by_month_client, by_cfop)
        # Guarda os resultados na sessão para reutilizar nos próximos cliques
        st.session_state["result"] = {
            "signature": signature, "df": df, "df_itens": df_itens,
            "errors": errors, "duplicates": duplicates, "total_found": total_found,
            "ignorados": ignorados,
            "agg": (by_month_client, by_month, by_client),
            "cfop": (by_cfop, by_month_cfop), "xlsx": xlsx, "pdf": pdf,
        }

    if ignorados:
        st.info(f"{ignorados:,} arquivo(s) com \"recebidos\" no nome não foram lidos.".replace(",", "."))

    if total_found > MAX_FILES:
        st.warning(
            f"Foram encontrados {total_found:,} XMLs, mas o limite por lote é {MAX_FILES:,}. "
            f"Somente os primeiros {MAX_FILES:,} foram processados.".replace(",", ".")
        )

    if not df.empty:
        st.success(f"{len(df):,} nota(s) processada(s) com sucesso.".replace(",", "."))
        linhas_emit = emitentes_lista(df)
        for linha in linhas_emit:
            st.caption(f"<U+1F3E2> {linha} (vendas a consumidor final)")
        if not linhas_emit:
            st.caption("Nenhuma venda a consumidor final encontrada no lote.")
        if duplicates:
            st.info(f"{duplicates:,} arquivo(s) duplicado(s) (mesma chave de acesso) foram ignorados."
                    .replace(",", "."))
        if errors:
            with st.expander(f"Ver {len(errors)} arquivo(s) não processado(s)"):
                for filename, error in errors[:500]:
                    st.warning(f"{filename}: {error}")
                if len(errors) > 500:
                    st.caption(f"... e mais {len(errors) - 500} erro(s).")

        # Indicadores no topo da tela
        c1, c2, c3, c4, c5 = st.columns(5)
        c1.metric("Notas processadas", f"{len(df):,}".replace(",", "."))
        c2.metric("Faturamento total", brl(df["Faturamento (R$)"].sum()))
        c3.metric("ICMS total", brl(df["ICMS (R$)"].sum()))
        c4.metric("Clientes", f"{df['Documento Cliente'].nunique():,}".replace(",", "."))
        c5.metric("CFOPs distintos", f"{by_cfop['CFOP'].nunique():,}".replace(",", "."))

        st.subheader("Visão geral")
        tab0, tab1, tab2, tab3, tab4 = st.tabs([
            "Soma por CFOP", "Por mês e cliente", "Consolidado mensal",
            "Por cliente", "Notas detalhadas",
        ])
        with tab0:
            st.dataframe(by_cfop, use_container_width=True, hide_index=True)
            if not by_cfop.empty:
                st.bar_chart(by_cfop.set_index("CFOP")[["Valor Itens (R$)"]])
                with st.expander("CFOP por mês"):
                    st.dataframe(by_month_cfop, use_container_width=True, hide_index=True)
        with tab1:
            st.dataframe(by_month_client, use_container_width=True, hide_index=True)
        with tab2:
            st.dataframe(by_month, use_container_width=True, hide_index=True)
            if not by_month.empty:
                st.bar_chart(by_month.set_index("Mês/Ano")[["Faturamento (R$)"]])
        with tab3:
            st.dataframe(by_client, use_container_width=True, hide_index=True)
        with tab4:
            st.dataframe(df, use_container_width=True, hide_index=True)

        # Botões de download do Excel e do PDF
```

```
        st.subheader("Exportar relatórios")
        d1, d2 = st.columns(2)
        with d1:
            st.download_button(
                "⬇ Baixar relatório Excel",
                data=xlsx, file_name="relatorio_apuracao_xml.xlsx",
                mime="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
                use_container_width=True
            )
        with d2:
            st.download_button(
                "⬇ Baixar relatório PDF",
                data=pdf, file_name="relatorio_apuracao_xml.pdf",
                mime="application/pdf", use_container_width=True
            )
    elif errors:
        st.error("Nenhum XML válido foi processado.")
        for filename, error in errors[:200]:
            st.warning(f"{filename}: {error}")
    else:
        st.error("Nenhum XML encontrado nos arquivos enviados.")
else:
    st.info("Envie arquivos XML (ou ZIP) para visualizar os dados e habilitar os relatórios.")
    st.markdown("""
    **O aplicativo gera:**
    - Relatório Excel com abas de soma por CFOP, CFOP por mês, mês/cliente, consolidado mensal, consolidado por cliente, notas e itens detalhados.
    - Relatório PDF com indicadores gerais, soma por CFOP e consolidação por mês e cliente.
    - Indicadores de faturamento, ICMS, PIS, COFINS, quantidade de notas, clientes e CFOPs.
    - Lotes de até 8.000 XMLs (envie preferencialmente em um ZIP).
    """)
```